Trabalho MATA76 - API Chamados

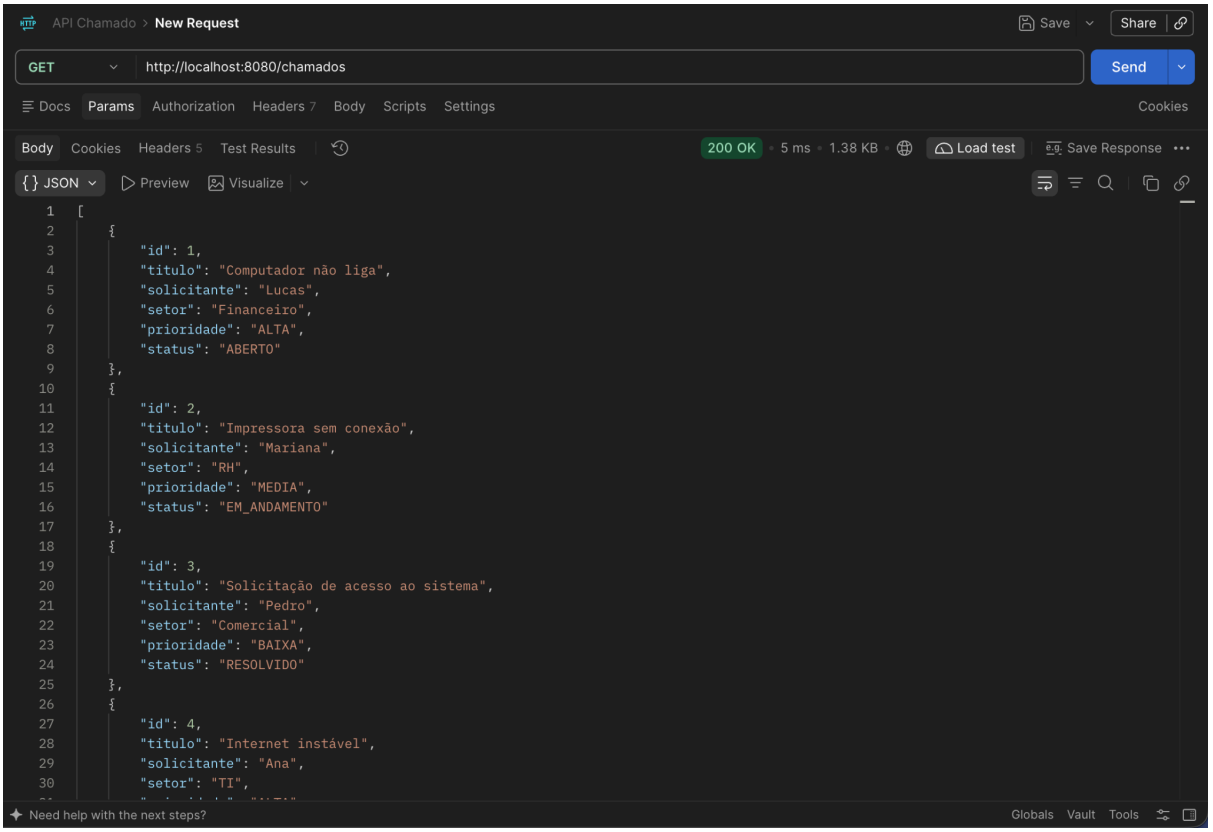
Equipe: Lucas Moreno dos Santos Oitabem, Daniel Jesus da Silva Araújo e Iago de Araújo Reis

Repositório: https://github.com/lucasmoreno01/api-chamados

Evidencias dos Testes API de Chamados MATA76

Os testes foram realizados utilizando o Postman, com o objetivo de verificar o funcionamento dos endpoints da API REST de gerenciamento de chamados. Foram realizadas requisições dos métodos GET e POST, verificando consultas, filtros, buscas, cadastro de novos registros e situações em que não existem resultados correspondentes.

1. Listar todos os chamados — GET /chamados



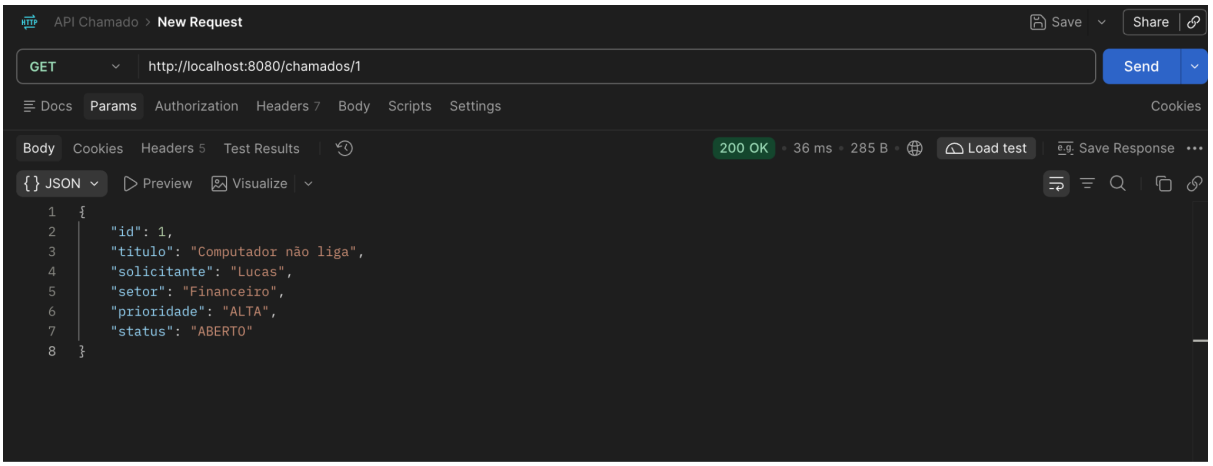
Foi realizada uma requisição GET para o endpoint ***/chamados***, com o objetivo de consultar todos os chamados cadastrados na aplicação.

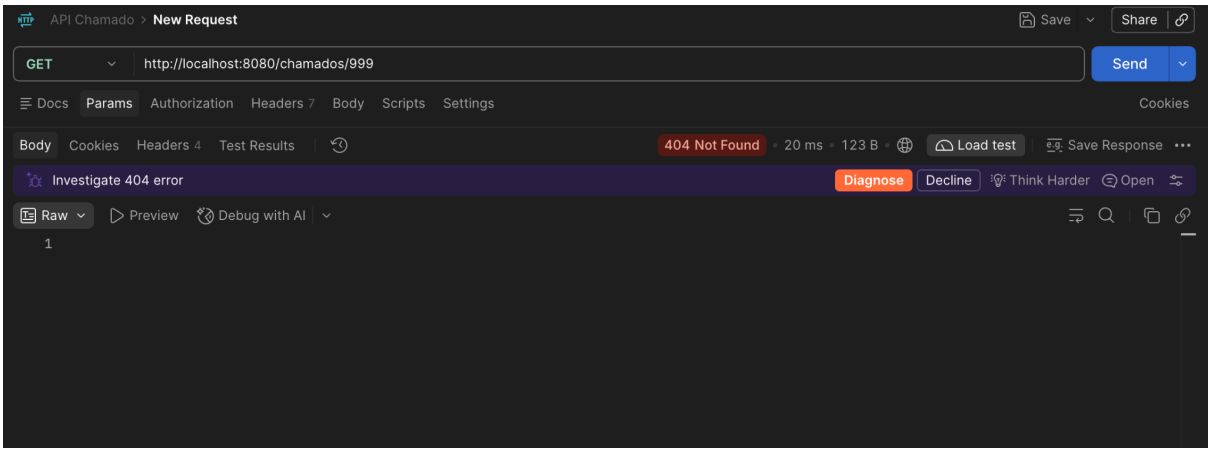
A requisição retornou uma lista em formato JSON contendo os chamados fictícios previamente definidos na coleção em memória. Cada registro apresentou informações como identificador, título, solicitante, setor, prioridade e status.

O teste confirmou que a API consegue recuperar e apresentar todos os registros disponíveis.

Resultado esperado: HTTP 200 OK, com a lista de chamados.

2. Consultar chamado por ID — ***GET /chamados/1***





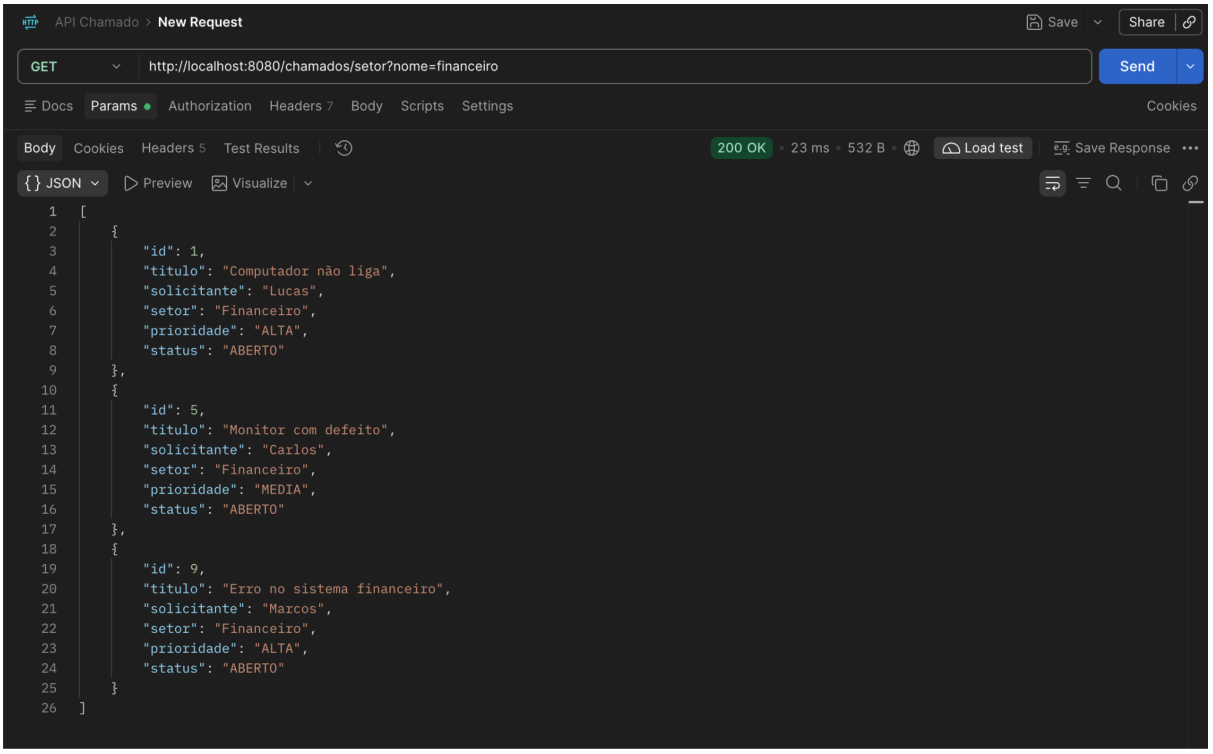
Foi realizada uma requisição GET para ***`/chamados/1`,*** informando o identificador do chamado diretamente na URL.

A API utilizou o ID recebido para procurar um registro correspondente na lista de chamados. Como foi encontrado um chamado com identificador 1, os dados desse registro foram retornados em formato JSON.

O teste também permite verificar o comportamento da aplicação quando um identificador não existe. Ao consultar `/chamados/999`, por exemplo, a API retorna HTTP 404 Not Found, indicando que nenhum chamado foi encontrado com aquele ID.

Resultado esperado: HTTP 200 OK quando o chamado existe e HTTP 404 Not Found quando não existe.

3. Filtrar chamados por setor — ***GET /chamados/setor?nome=financeiro***



Foi realizada uma requisição GET para ***`/chamados/setor?nome=financeiro`,*** utilizando o parâmetro **`nome`** para informar o setor desejado.

A aplicação percorreu a lista de chamados e selecionou somente aqueles cujo setor corresponde ao valor informado. A comparação utiliza o método **`equalsIgnoreCase()`**, permitindo realizar buscas sem diferenciar letras maiúsculas e minúsculas.

Dessa forma, mesmo informando `financeiro` em letras minúsculas, a API consegue localizar registros cadastrados com o setor `Financeiro`.

Também foi possível verificar que, ao pesquisar um setor inexistente, como `juridico`, a API retorna uma lista vazia.

Resultado esperado: HTTP 200 OK, contendo apenas os chamados do setor informado ou uma lista vazia quando não existem correspondências.

4. Filtrar chamados por prioridade — ***GET /chamados/prioridade?nivel=alta***

Foi realizada uma requisição GET para ***`/chamados/prioridade?nivel=alta`,*** utilizando o parâmetro `nivel` para definir a prioridade desejada.

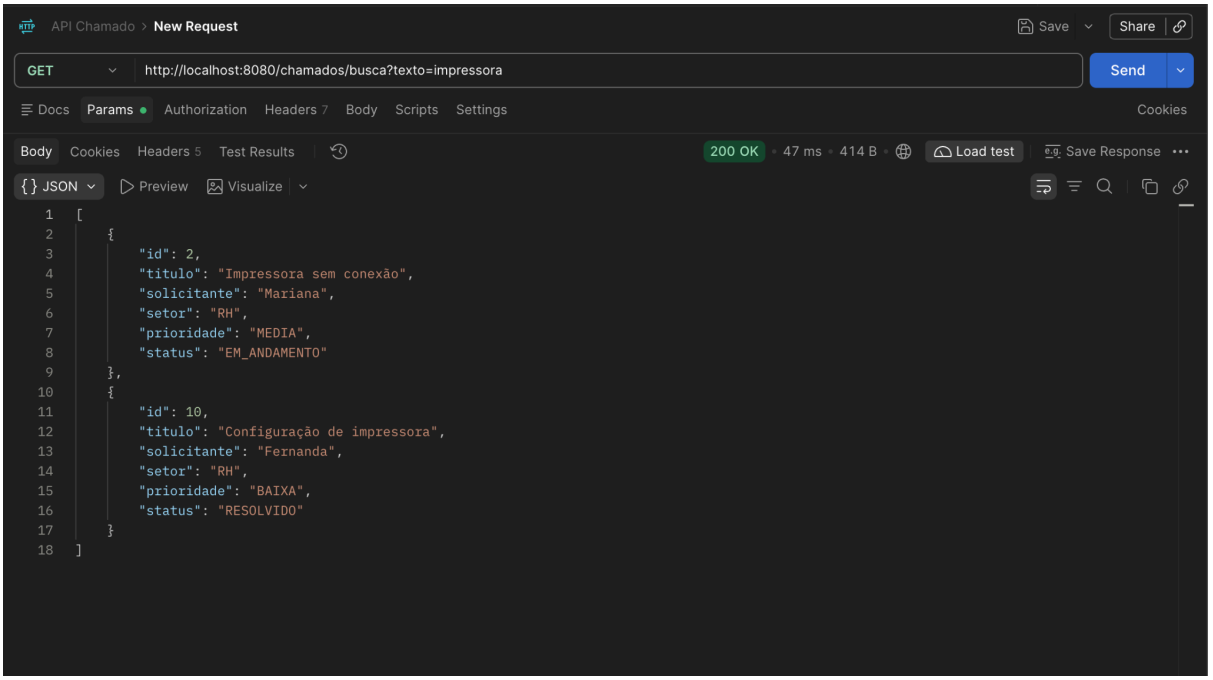
A aplicação converte o valor recebido para letras maiúsculas e utiliza **`Priority.valueOf()`** para identificar a prioridade correspondente ao enum **`Priority`.**

Após essa conversão, são retornados apenas os chamados que possuem a prioridade informada.

Também foi testado o tratamento de valores inválidos. Caso seja informada uma prioridade diferente das opções permitidas **(BAIXA, MEDIA ou ALTA),** a aplicação retorna **HTTP 400** Bad Request, apresentando uma mensagem informando os valores aceitos.

Resultado esperado: HTTP 200 OK para prioridades válidas e HTTP 400 Bad Request para prioridades inválidas.

5. Buscar chamados pelo título — ***GET /chamados/busca?texto=impressora***



Foi realizada uma requisição GET para ***`/chamados/busca?texto=impressora`,*** utilizando o parâmetro **`texto`** para pesquisar chamados que contenham a palavra informada em seus títulos.

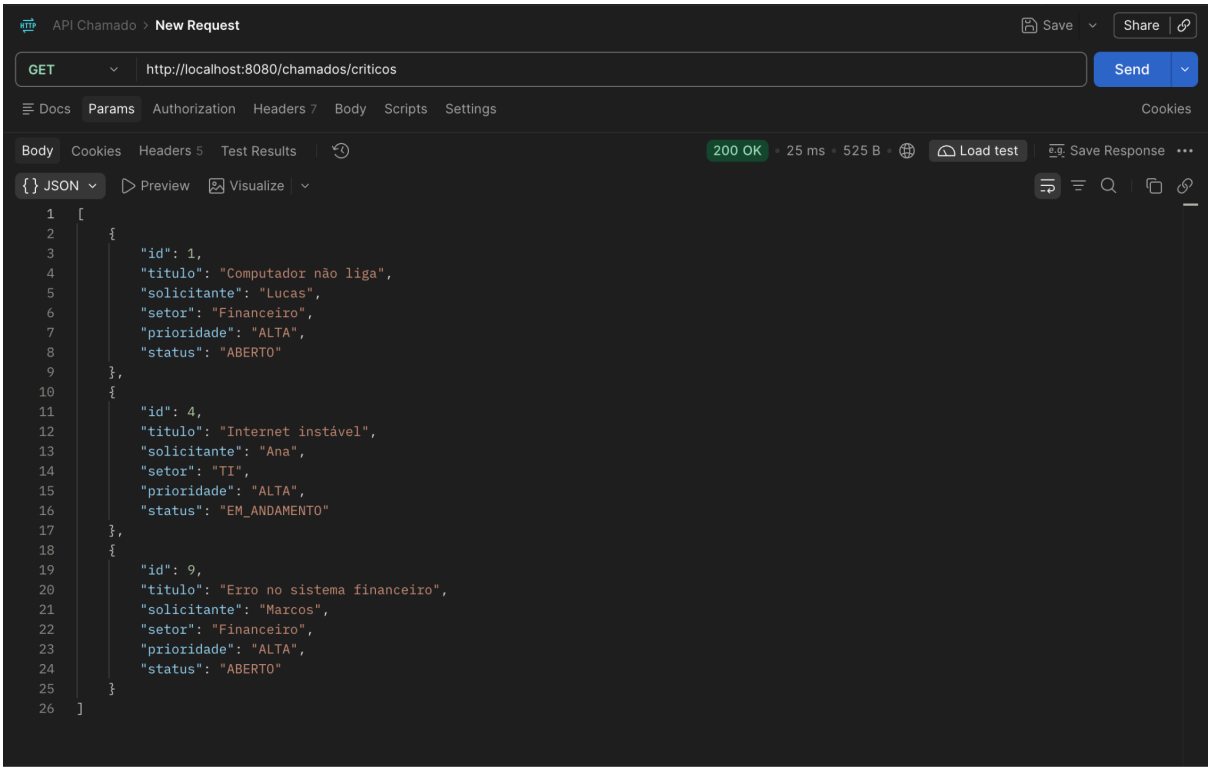
A aplicação percorreu os registros e utilizou o método **`contains()`** para identificar títulos que possuem o termo pesquisado. Antes da comparação, os textos são convertidos para letras minúsculas, permitindo realizar buscas sem diferenciar maiúsculas e minúsculas.

Assim, a pesquisa também consegue localizar títulos que contenham a palavra **`Impressora`**, mesmo quando o parâmetro é enviado como **`impressora`**.

Ao pesquisar um termo que não aparece em nenhum título, como **`notebookxyz`**, a API retorna uma lista vazia.

Resultado esperado: HTTP 200 OK, contendo os chamados encontrados ou uma lista vazia quando não existem correspondências.

6. Listar chamados críticos — ***GET /chamados/criticos***



Foi realizada uma requisição GET para **`/chamados/criticos`**, com o objetivo de identificar os chamados que necessitam de maior atenção da equipe de suporte.

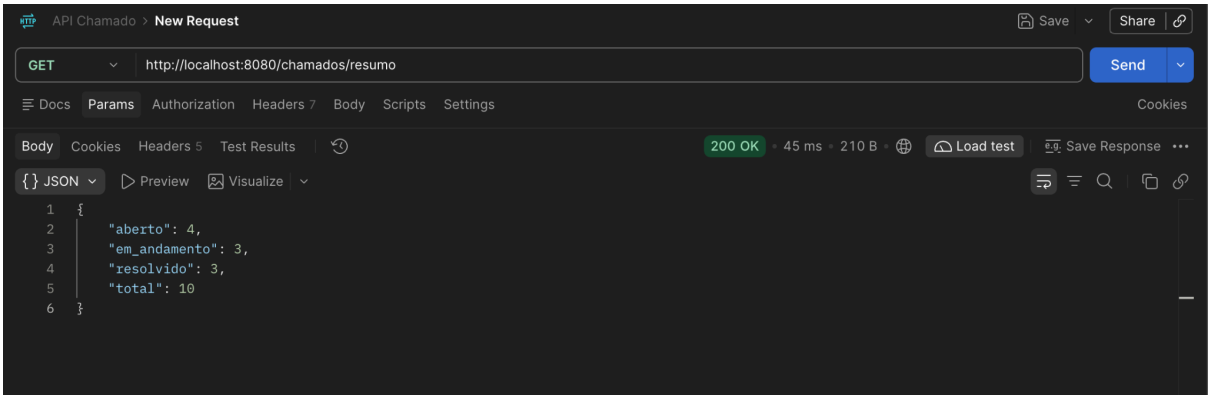
Nesse endpoint, a aplicação combina dois critérios de filtragem: o chamado deve possuir prioridade ALTA e seu status não pode ser RESOLVIDO.

Dessa forma, são apresentados tanto os chamados que ainda estão ABERTOS quanto aqueles que estão **EM_ANDAMENTO**, desde que possuam prioridade alta.

O teste confirmou que chamados de prioridade alta já resolvidos não aparecem no resultado, pois não representam pendências de atendimento.

Resultado esperado: HTTP 200 OK, contendo somente os chamados de prioridade ALTA que ainda não foram resolvidos.

7. Consultar resumo dos chamados por status — GET ***/chamados/resumo***



Foi realizada uma requisição GET para **`/chamados/resumo`**, com o objetivo de obter uma visão geral da distribuição dos chamados conforme seus estados de atendimento.

A aplicação percorre os valores definidos no enum **`Status`** e contabiliza quantos chamados pertencem a cada situação: ABERTO, EM_ANDAMENTO e RESOLVIDO.

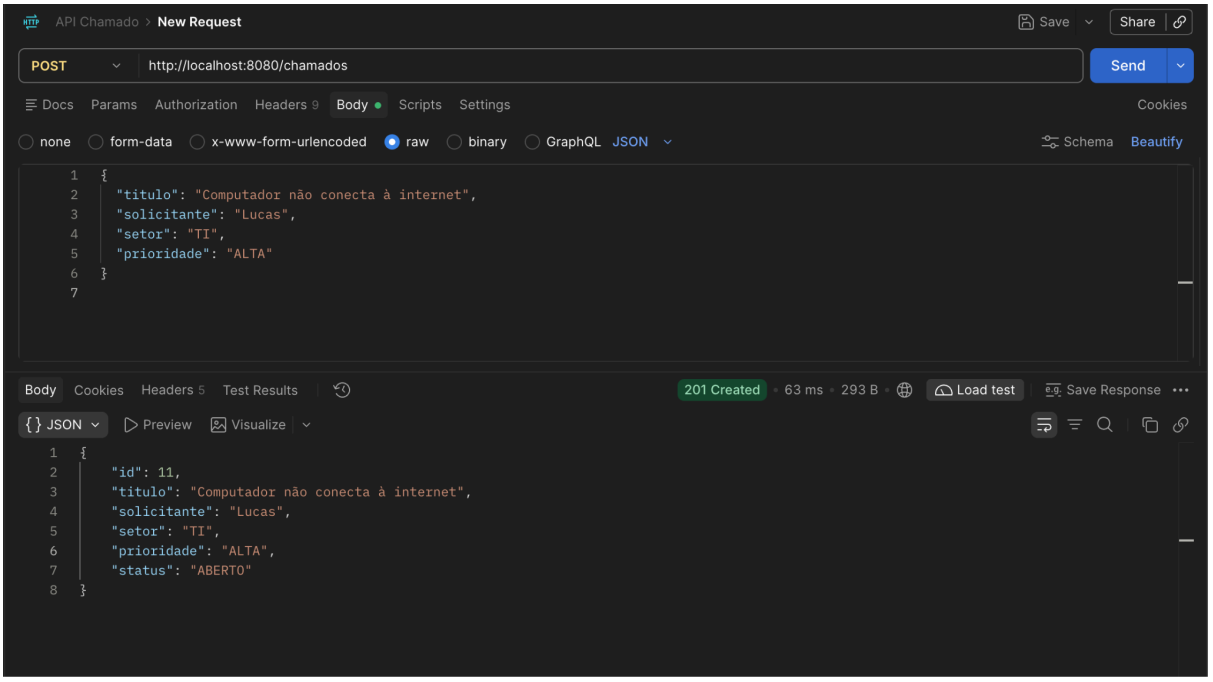
Para organizar os resultados, foi utilizado um **`EnumMap`**, que associa cada status à respectiva quantidade de chamados.

A resposta apresenta um JSON com as quantidades encontradas para cada status.

Resultado esperado: HTTP 200 OK, com um JSON contendo as quantidades de chamados por status.

Observação: a implementação atual retorna os status como chaves do JSON, e não inclui uma propriedade com o total geral. Portanto, para atender integralmente ao enunciado, será necessário adicionar essa informação ao endpoint.

8. Cadastrar novo chamado — POST **`/chamados`**



Foi realizada uma requisição POST para **`/chamados`,** enviando um objeto JSON no corpo da requisição com as informações de um novo chamado.

Exemplo do JSON utilizado:

```
None
{
  "titulo": "Computador não conecta à internet",
  "solicitante": "Lucas",
  "setor": "TI",
  "prioridade": "ALTA"
}
```

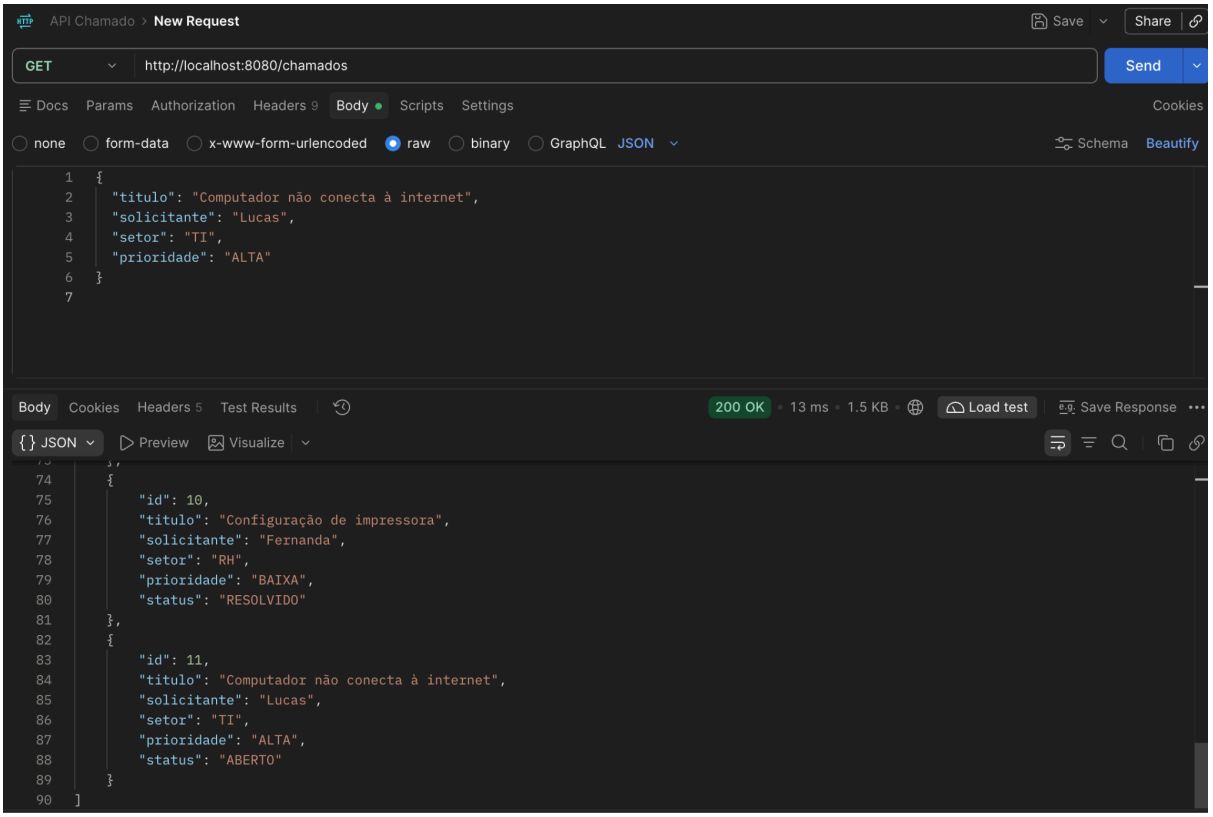
A aplicação recebeu os dados por meio da anotação `@RequestBody` e realizou verificações para garantir que os campos título, solicitante, setor e prioridade fossem informados corretamente.

Após validar os dados, a aplicação gerou automaticamente um novo identificador, definiu o status inicial como ABERTO e adicionou o chamado à lista armazenada em memória.

A resposta retornou os dados do chamado cadastrado, juntamente com o identificador gerado automaticamente.

Resultado esperado: HTTP 201 Created, indicando que o novo chamado foi cadastrado com sucesso.

9. Verificar chamado cadastrado — ***GET /chamados/***



Após realizar o cadastro do novo chamado, foi feita uma nova requisição GET utilizando o identificador retornado pela operação POST.

O objetivo foi verificar se o registro havia sido adicionado corretamente à coleção em memória.

A aplicação localizou o chamado pelo ID e retornou todas as suas informações, permitindo confirmar que o cadastro foi realizado e que os dados permaneceram disponíveis durante a execução da aplicação.

Resultado esperado: HTTP 200 OK, contendo os dados do chamado cadastrado anteriormente.